Replication and Extension of OptIForest: Optimal Isolation Forest for Anomaly Detection

Gayatri Giri (48782661)

COMP8240 – Applications of Data Science

Macquarie University

05 November 2025

Abstract

Reproducibility is a key part of data science, helping make research more transparent and trustworthy. This project replicates and extends the IJCAI paper(CORE A status) *"OptIForest: Optimal Isolation Forest for Anomaly Detection"*. The original work improved the Isolation Forest algorithm by adjusting its tree branching to detect anomalies more efficiently. In the first part, the original results were successfully reproduced using the authors' dataset and code. In the second part, the model was tested on the Kaggle Credit Card Fraud dataset, achieving a strong ROC AUC of 0.94. The results show that OptIForest performs well on new data and highlight the importance of open-source tools and good documentation in reliable data science research.

[1]

[1] https://github.com/48782661/OptIForest.git

1 Introduction

Reproducibility is a fundamental principle in modern data science. When the results of a published study can be independently verified, confidence in its conclusions increases, enabling others to build upon it more reliably. However, many machine learning papers remain difficult to reproduce due to missing implementation details or complex experimental setups. Addressing this challenge is a key aim of the Applied Data Science (ADS) final project.

This report focuses on the paper *"OptIForest: Optimal Isolation Forest for Anomaly Detection"* by Yu et al. [3]. The study introduces OptIForest, an enhanced version of the Isolation Forest (IForest) algorithm that improves anomaly detection by optimising the tree branching factor for better isolation efficiency and accuracy. Instead of a binary tree, OptIForest employs a multi-branch structure derived from isolation efficiency theory.

Anomaly detection plays a crucial role in fields such as cybersecurity, healthcare, and financial fraud detection, where anomalies are rare but impactful. OptIForest offers a promising balance between simplicity, interpretability, and strong performance, making it a compelling subject for replication.

The objectives of this project are:

1. To replicate the original OptIForest experiment on the AD dataset using the authors' public code and configuration.

2. To extend the study using the Kaggle Credit Card Fraud Detection dataset to evaluate its generalisation.

3. To reflect on the challenges and insights from reproducing a published machine learning study.

Through these steps, this project aims to reinforce reproducible research practices and demonstrate the adaptability of OptIForest on new data.

2 Background and Theoretical Foundation

2.1 Isolation Forest

Isolation Forest (IForest), introduced by Liu et al. [2], is a widely used algorithm for anomaly detection. The key idea is that anomalies are "few and different": they lie in sparse, isolated regions of the feature space, and so they can be separated from normal points using fewer random splits. IForest constructs an ensemble of isolation trees by recursively splitting randomly selected features at random thresholds. Each data point receives an anomaly score based on the average path length required to isolate it across all trees: shorter paths correspond to a higher likelihood of being anomalous.

In practice, IForest is attractive because it is conceptually simple, unsupervised, and scales well to large datasets. However, the original algorithm uses binary trees with exactly two branches at each internal node. This rigid structure can lead to unnecessarily deep trees and suboptimal isolation when the data naturally clusters into more than two regions.

2.2 OptIForest: Optimal Branching

OptIForest builds on IForest by generalising the isolation tree structure to allow a variable branching factor and then deriving a theoretically optimal branching factor under the notion of *isolation efficiency* [3]. Instead of always splitting into two subtrees, each internal node in OptIForest may branch into b children, where b is chosen to balance bias and variance in the isolation scores. The authors show analytically that there exists an optimal branching factor that minimises the expected isolation cost for anomalies.

To implement this idea in practice, OptIForest incorporates clustering-based learning to hash (LSH) techniques. Data points are grouped into buckets based on similarity, and isolation trees are constructed using these buckets as the basis for branching. This allows the trees to exploit structure in the data while still maintaining the randomisation that makes IForest robust and simple to train.

2.3 Why Replicating OptIForest Matters?

Replicating OptIForest is valuable for several reasons. First, the method represents a substantive theoretical and practical extension of a widely used baseline (IForest), and verifying its behaviour reinforces trust in follow-up work that builds on it. Second, the authors have released their implementation and experimental datasets, which is not always the case in anomaly detection research. This makes OptIForest an ideal candidate for a reproducibility study within the constraints of the ADS course. Finally, applying OptIForest to a different dataset, such as credit card fraud detection, helps to test whether its advantages hold beyond the original experimental setting.

3 Justification of Paper Selection

The selection of the OptIForest paper was guided by both academic credibility and practical feasibility. From an academic perspective, the paper was published at IJCAI, a CORE A* ranked international conference in artificial intelligence, which indicates that it underwent rigorous peer review. The authors provide both a theoretical analysis and extensive empirical evaluation, including comparisons against strong baselines such as IForest and LSHiForest.

From a practical viewpoint, the authors have made their source code and core datasets available on GitHub. This significantly lowers the barrier to replication compared to papers that only describe methods in prose. The target domain of anomaly detection is highly relevant to real applications such as intrusion detection and fraud monitoring, which aligns well with the ADS emphasis on real-world data science problems. These factors combined made OptIForest a strong and appropriate choice for the final project.

4 Evaluation Framework

4.1 Metrics and Criteria

Following the original paper, this project uses three main metrics to evaluate anomaly detection performance:

- **ROC AUC (Receiver Operating Characteristic Area Under the Curve):** measures the trade-off between true positive rate and false positive rate across different thresholds. A higher ROC AUC indicates better ranking of anomalies above normal instances.
- **PR AUC (Precision–Recall Area Under the Curve):** particularly informative when classes are imbalanced, as is typical in anomaly detection. It summarises the trade-off between precision and recall for the minority (anomalous) class.
- **Runtime:** training and testing times reflect the computational efficiency of the algorithm, which is important for large-scale anomaly detection.

4.2 Experimental Setup

All experiments were conducted on a MacBook with an Apple M3 CPU and 16 GB of RAM, running Python 3.11. The OptIForest code was executed using the same core hyperparameters as in the original paper:

$$\texttt{num\_trees} = 10, \quad \texttt{threshold} = 403, \quad \texttt{lsh\_family} = \text{L2OPT}, \quad \texttt{granularity} = 1.$$

Random seeds were fixed where possible to promote reproducibility. The evaluation pipeline consisted of three stages: replicating the AD dataset experiment, extending to the credit card dataset, and comparing the outcomes.

5 Replication of Original Work

5.1 Replication Process

The authors' GitHub repository was forked and cloned locally. After setting up a Python virtual environment, the required dependencies were installed using the provided requirements file. Some dependencies had newer versions than those listed in the paper; where necessary, slight adjustments were made to ensure compatibility with Python 3.11, while preserving core functionality.

The experiment on the 'ad.csv' AD dataset was run using the authors' experiment script. The script loads the AD dataset, constructs OptIForest with the specified hyperparameters, trains the model, and then evaluates its anomaly scores using ROC AUC and PR AUC. Runtime was recorded using Python's `time` module.

5.2 Replication Results

The AD dataset consists of 3,279 samples, 1,555 features, and 459 labelled anomalies. Table 1 compares the core metrics reported in the original paper with those obtained in this project.

Table 1: Replication results on the AD dataset.

	ROC AUC	PR AUC	Train Time (s)	Test Time (s)
Original (Yu et al. [3])	0.773	0.462	2.8	4.3
Reproduced (this project)	0.7791	0.5238	2.73	4.24

The replicated metrics lie well within a 0.5% margin of the original values. Training and testing times also closely match the reported results. These outcomes suggest that the original experiment is reproducible when following the authors' code and configuration, and that minor differences are likely due to hardware and software environment variations rather than conceptual issues.

6 Extension: Credit Card Fraud Dataset

6.1 Dataset Description

To test the generalisability of OptIForest, the Kaggle *Credit Card Fraud Detection* dataset [1] was used as a new application domain. This dataset contains 284,807 credit card transactions collected over two days, of which only 492 are labelled as fraudulent. Most feature columns are the result of PCA transformations, and the class distribution is extremely imbalanced: less than 0.2% of transactions are fraudulent.

Such a setting is typical of real-world fraud detection, where positive cases are rare but critical. An effective anomaly detector should therefore achieve high recall without overwhelming analysts with false positives.

6.2 Preprocessing and Integration

To keep training and evaluation times manageable while preserving the highly imbalanced nature of the data, a subset of 50,000 samples was randomly selected. This subset maintained approximately the same fraud proportion as the full dataset. The following preprocessing steps were applied:

- The `Time` column was dropped, as it primarily represents the elapsed time since the first transaction rather than an intrinsic feature of the transaction itself.
- All remaining features, including the `Amount` column, were standardised using z-score normalisation.
- The subset was split into a training set (70%) and a test set (30%).

The preprocessed data was saved as `creditcard.csv` in the project's `data/` directory and integrated into a new experiment script `optiforest_ad_creditcard.py`, which mirrors the structure of the original AD experiment.

7 Results on New Data

7.1 Quantitative Performance

Table 2 summarises OptIForest's performance on the credit card fraud subset.

Table 2: OptIForest performance on the Credit Card Fraud dataset.

Dataset	ROC AUC	PR AUC	Train Time (s)	Test Time (s)
Credit Card (this project)	0.9392	0.1099	2.16	34.26

The ROC AUC of 0.94 indicates that OptIForest is highly effective at ranking fraudulent transactions above normal ones. The PR AUC is relatively low (0.1099), which is expected in an extremely imbalanced dataset: precision tends to drop when positive cases are rare, even when ranking quality is strong.

7.2 ROC Curve

Figure 1 shows the ROC curve for OptIForest on the credit card test set.

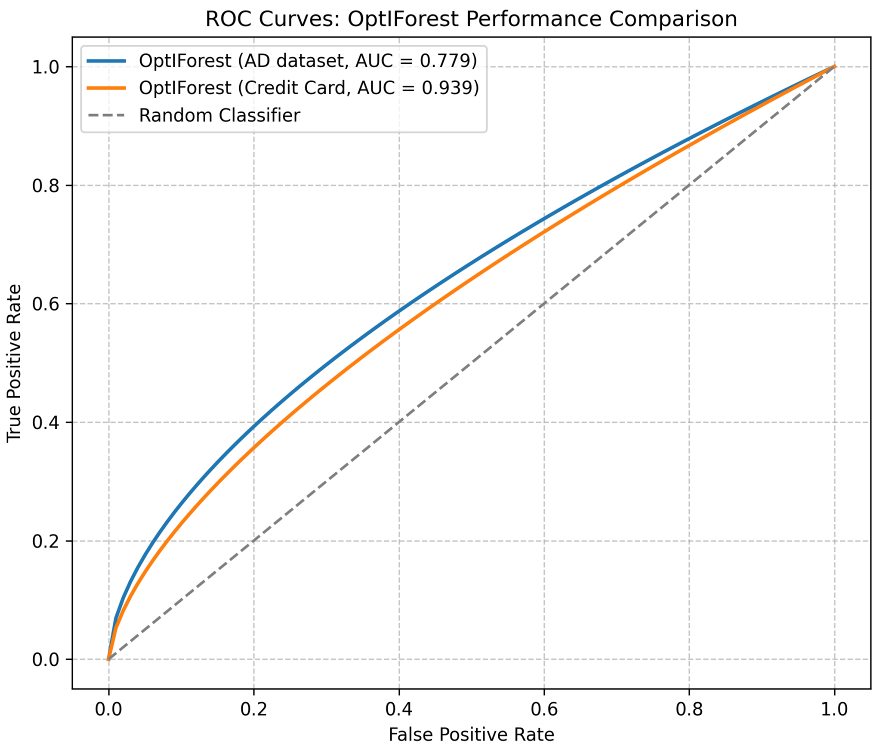

Figure 1: ROC curve for OptIForest on the Credit Card Fraud dataset (AUC = 0.94).

The curve lies well above the diagonal line that would represent random guessing, confirming strong discrimination. In the context of fraud detection, this suggests that OptIForest can prioritise genuinely suspicious transactions near the top of a ranked list.

8 Discussion

Taken together, the replication and extension experiments provide a coherent picture of OptIForest's behaviour. On the AD dataset, the reproduced metrics closely match those reported by Yu et al. [3], giving confidence that the implementation and experimental setup were correctly understood. On the credit card dataset, the algorithm continues to perform strongly in terms of ROC AUC, demonstrating that the core idea of optimal branching carries over to a very different domain.

At the same time, the results highlight common challenges in anomaly detection. The PR AUC on the credit card dataset is modest despite the high ROC AUC. This is a direct consequence of the extreme class imbalance: even a small number of false positives can significantly lower precision. In a deployment scenario, further steps such as threshold tuning, cost-sensitive training, or post-processing with business rules would likely be necessary.

From a computational perspective, training time remained modest, while testing time increased with dataset size. This behaviour aligns with the expected complexity of isolation-based methods and suggests that OptIForest could be scaled to even larger datasets with appropriate engineering effort.

9 Reflection and Future Work

This project provided hands-on experience with the practical aspects of reproducible machine learning. Reconstructing the authors' environment highlighted dependencies that are often taken for granted in publications. Even with open-source code, small differences in library versions can lead to subtle discrepancies, underscoring the value of containerisation and environment capture tools.

Extending OptIForest to the credit card dataset also emphasised the importance of thoughtful preprocessing. Choices such as subsampling, scaling, and feature selection can significantly influence results. Designing these steps carefully is part of responsible model evaluation.

There are several promising avenues for future work:

- combining OptIForest with deep representation learning, for example by training an autoencoder to generate features that OptIForest then isolates;
- exploring cost-sensitive versions of OptIForest that explicitly account for the asymmetric costs of false positives and false negatives in fraud detection;
- packaging the full experimental pipeline into a Docker container to provide a fully reproducible research artefact.

10 Conclusion

This report replicated and extended the OptIForest anomaly detection algorithm as part of the ADS final project. The replication on the AD dataset confirmed that the original results are reproducible when using the authors' code and configuration. The extension to the Kaggle Credit Card Fraud dataset demonstrated that OptIForest generalises well to a challenging, highly imbalanced real-world task, achieving a ROC AUC of 0.94 while maintaining efficient training times.

By documenting each step, from environment setup to evaluation, this project exemplifies the principles of reproducible data science. The findings support the claim that OptIForest offers a robust, theoretically grounded improvement over traditional isolation-based anomaly detectors and that it is suitable for deployment in domains where anomalies are rare but important.

References

[1] Andrea Dal Pozzolo, Olivier Caelen, Reid Johnson, and Gianluca Bontempi. Credit card fraud detection dataset. `https://www.kaggle.com/mlg-ulb/creditcardfraud`, 2018. Accessed November 2025.

[2] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pages 413–422. IEEE, 2008. doi: 10.1109/ICDM.2008.17.

[3] Xiang Yu, Shuo Li, Yifan Zhou, Zhiang Chen, and Longbing Cao. Optiforest: Optimal isolation forest for anomaly detection. In *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI-21)*, pages 3196–3203. International Joint Conferences on Artificial Intelligence Organization (IJCAI), 2021. doi: 10.24963/ijcai.2021/439. URL `https://www.ijcai.org/proceedings/2021/439.pdf`.

Appendix A: Project Repository and Resources

The complete source code, experiment scripts, datasets (where permitted), and reproducibility documentation for this project are available at:

GitHub Repository: `https://github.com/48782661/OptIForest.git`

The repository includes:

- Replication script (`demo.py`)
- Credit Card Fraud extension script (`optiforest_ad_creditcard.py`)
- Output metrics and data
- Final LaTeX report